

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	HARIHARAN G	Reg. No.:	192424156
Section / Batch:	B.Tech AIDS / 3 rd Year	Date:	20/08/2026

Code of Conduct

I HARIHARAN G (1924224156) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ HARIHARAN G

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

Annexure A – Data Interpretation

1. Frame Semantics in Banking Customer-Service Systems

A digital banking platform uses semantic frames to understand customer requests. The NLP system identifies the **action, agent, object, and destination** involved in each request.

Customer Query	Generated Semantic Frame
"Transfer ₹20,000 from my savings account to my son's account."	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)
"Block my debit card immediately."	BLOCK(DebitCard, Customer)
"Send my transaction statement to my email."	SEND(TransactionStatement, Email, Customer)
"Increase my daily withdrawal limit."	MODIFY(WithdrawalLimit, Customer)

Additional transaction logs show:

Query ID	Actual User Intent	System Interpretation
B1	Transfer money to another account	Transfer money
B2	Block debit card	Block debit card
B3	Receive transaction statement by email	Send statement to email
B4	Increase withdrawal limit	Decrease withdrawal limit

Tasks:

1. Analyze the semantic frame of each banking request and identify the relationship between the action and its participants.
2. Identify the query in which the semantic interpretation is incorrect and explain why.
3. Analyze how incorrect identification of semantic roles can lead to an incorrect banking operation.
4. Recommend a frame-based semantic analysis approach to improve the reliability of banking conversational systems.

2. First-Order Predicate Calculus for Smart Agriculture

A smart agricultural system monitors crop fields using sensors and logical rules.

Field Data:

Field	Soil Condition	Irrigation Status	Crop
F1	Dry	Available	Rice
F2	Wet	Available	Wheat
F3	Dry	Unavailable	Maize
F4	Wet	Unavailable	Rice

The system uses the following predicate rules:

- $\text{Dry}(x) \wedge \text{IrrigationAvailable}(x) \rightarrow \text{NeedsWater}(x)$
- $\text{NeedsWater}(x) \rightarrow \text{Irrigate}(x)$
- $\text{Wet}(x) \rightarrow \neg \text{NeedsWater}(x)$
- $\text{IrrigationUnavailable}(x) \rightarrow \neg \text{Irrigate}(x)$
- $\text{Rice}(x) \wedge \text{NeedsWater}(x) \rightarrow \text{PriorityIrrigation}(x)$

Tasks:

1. Represent the field observations using First-Order Predicate Calculus.
2. Using the given rules, determine which fields require irrigation.
3. Determine whether F1 and F3 should be treated differently by the automated irrigation controller.
4. Analyze whether predicate logic alone is sufficient for agricultural decision support when sensor information is incomplete or uncertain.

3. Word Sense Disambiguation in a Travel Recommendation System

A travel platform receives search queries containing ambiguous words. The recommendation engine uses surrounding words and user interactions to determine the intended meaning.

Search Query	Possible Sense 1	Possible Sense 2
"Amazon tour"	Amazon rainforest	Amazon company
"Safari booking"	Wildlife tour	Web browser
"Java vacation"	Java island	Java programming language
"Cruise package"	Ship journey	Cruise software/project

The system records the following user interactions:

Query	User Interaction
Amazon tour	Viewed rainforest trekking packages
Safari booking	Selected wildlife resort packages
Java vacation	Viewed hotels in Bali and Java
Cruise package	Viewed Mediterranean ship packages

However, another user searches: **"Safari download for Windows"** and clicks a browser download page.

Tasks:

1. Determine the intended sense of the ambiguous terms using the available context.
2. Identify the lexical and contextual cues that distinguish the possible senses.
3. Analyze why the same word may require different interpretations for different users.
4. Design an industrial-scale WSD strategy using query context, user history, embeddings, and click behaviour to improve travel recommendations.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

A legal NLP system analyzes sentences from contracts and assigns semantic roles based on their syntactic structures.

The system produces the following analyses:

Legal Sentence	Parse Structure	Generated Semantic Interpretation
"The supplier delivered the equipment to the buyer."	Subject–Verb–Object	Supplier = Agent, Equipment = Object, Buyer = Recipient
"The buyer received the equipment from the supplier."	Subject–Verb–Object	Buyer = Agent, Equipment = Object, Supplier = Recipient
"The company terminated the contract after the violation."	Subject–Verb–Object	Company = Agent, Contract = Object
"The contract was terminated by the company."	Passive Construction	Contract = Agent, Company = Object

Tasks:

1. Analyze how syntactic structure, including active and passive constructions, influences semantic-role assignment.
2. Identify the incorrect semantic-role assignments in the given interpretations.
3. Explain how confusing grammatical subject with semantic agent can affect automated legal information extraction.
4. Recommend a syntax-driven semantic analysis architecture that can correctly handle active voice, passive voice, prepositional phrases, and long-distance dependencies in legal documents.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation

		structure, visuals, and terminology			and difficult to understand
--	--	-------------------------------------	--	--	-----------------------------

Q. No. / Criteria Level	Understanding of Data	Analysis	Application of Concepts	Conclusion	Clarity	Sub. Total	Total %
1	4	4	4	4	3	19	92
2	4	4	4	3	4	19	
3	4	4	3	4	3	18	
4	3	4	4	3	4	18	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 20/08/2026	Date: _____	Date: _____

1. Frame Semantics in Banking Customer-Service Systems

1. Semantic frame analysis

A **semantic frame** represents an action or event together with its participants and their roles.

Query	Frame	Action	Participants and Roles
“Transfer ₹20,000 from my savings account to my son's account.”	TRANSFER(SourceAccount, Amount, DestinationAccount, Customer)	Transfer	Customer = initiator, Savings Account = source, ₹20,000 = amount, Son's Account = destination
“Block my debit card immediately.”	BLOCK(DebitCard, Customer)	Block	Customer = requester, Debit Card = object
“Send my transaction statement to my email.”	SEND(TransactionStatement, Email, Customer)	Send	Customer = requester, Transaction Statement = object, Email = destination
“Increase my daily withdrawal limit.”	MODIFY(WithdrawalLimit, Customer)	Modify	Customer = requester, Withdrawal Limit = object, Increase = requested modification

The frame identifies **what action is requested and which entities participate in that action**.

2. Incorrect semantic interpretation

B4 is incorrect.

- Actual intent: **Increase withdrawal limit**
- System interpretation: **Decrease withdrawal limit**

The system correctly identified the object, *withdrawal limit*, but incorrectly interpreted the action direction. The word “**increase**” explicitly indicates that the value should become larger.

B1, B2, and B3 have interpretations consistent with their actual intents.

3. Effect of incorrect semantic roles

Incorrect semantic-role identification can result in a **wrong financial operation**.

For example, if:

“Increase my daily withdrawal limit”

is interpreted as:

“Decrease my daily withdrawal limit”

the banking system may actually reduce the customer's permitted withdrawal amount.

Similarly, incorrectly identifying the source and destination accounts in a transfer could potentially result in money being transferred to the wrong account. Therefore, semantic errors in banking systems can have serious financial and security consequences.

4. Recommended frame-based approach

A reliable banking conversational system should use a **frame-based semantic pipeline**:

User Query → Intent Detection → Entity Recognition → Semantic Role Assignment → Frame Construction → Validation → Banking Operation

The system should:

1. Detect the main action such as TRANSFER, BLOCK, SEND, or MODIFY.
2. Extract entities such as account numbers, amounts, cards, and email addresses.
3. Assign semantic roles such as **source, destination, object, amount, customer, and recipient**.
4. Detect modifiers such as **increase/decrease, immediately, and recurring**.
5. Validate the complete frame against banking rules before executing an operation.
6. Request confirmation for high-risk transactions.

This approach improves **accuracy, explainability, and transaction safety**.

2. First-Order Predicate Calculus for Smart Agriculture

1. Representing observations using FOPC

The field observations can be represented using predicates:

- Dry(x) — Field x has dry soil.
- Wet(x) — Field x has wet soil.
- IrrigationAvailable(x) — irrigation is available for field x.
- IrrigationUnavailable(x) — irrigation is unavailable.
- Rice(x) — field x contains rice.
- Wheat(x) — field x contains wheat.
- Maize(x) — field x contains maize.

The observations are:

F1

- Dry(F1)
- IrrigationAvailable(F1)
- Rice(F1)

F2

- Wet(F2)
- IrrigationAvailable(F2)
- Wheat(F2)

F3

- Dry(F3)
- IrrigationUnavailable(F3)
- Maize(F3)

F4

- Wet(F4)
- IrrigationUnavailable(F4)
- Rice(F4)

The rules are:

2. Fields requiring irrigation

F1

Given:

Dry(F1) and IrrigationAvailable(F1)

Therefore:

NeedsWater(F1)

Then:

Irrigate(F1)

Since F1 contains rice:

PriorityIrrigation(F1)

F1 requires irrigation and has priority.

F2

Wet(F2) gives:

$\neg$ NeedsWater(F2)

Therefore, **F2 does not require irrigation.**

F3

Dry(F3) suggests that it needs water, but irrigation is unavailable.

The first rule cannot be applied because:

IrrigationAvailable(F3) is false.

Therefore, NeedsWater(F3) cannot be derived from the supplied rules.

Also:

$\text{IrrigationUnavailable(F3)} \rightarrow \neg \text{Irrigate(F3)}$

Hence, **F3 cannot be irrigated automatically.**

F4

$\text{Wet(F4)} \rightarrow \neg \text{NeedsWater(F4)}$

Therefore, **F4 does not require irrigation.**

Final result

Field	Needs Water	Irrigation Possible?	Controller Action
F1	Yes	Yes	Irrigate
F2	No	Yes	Do not irrigate
F3	Not derivable	No	Do not irrigate
F4	No	No	Do not irrigate

3. Should F1 and F3 be treated differently?

Yes.

Both F1 and F3 have **dry soil**, but their irrigation availability differs.

- **F1:** Dry + irrigation available → irrigation should be activated.
- **F3:** Dry + irrigation unavailable → irrigation cannot be activated.

Therefore, the controller should not treat soil condition alone as sufficient. It must also consider **resource availability**.

F1 receives irrigation, while F3 may require an alternative action such as notifying the farmer or activating another available water source.

4. Is predicate logic sufficient?

No. Predicate logic alone is not sufficient for real-world agricultural decision support.

FOPC works well with clear facts, such as:

Dry(F1)

However, real sensors may provide uncertain or incomplete information:

- Soil moisture sensor may malfunction.
- A sensor may report 45% moisture with uncertainty.
- Weather forecasts may be unreliable.
- Irrigation availability may change.
- Different sensors may provide conflicting measurements.

Classical predicate logic generally represents facts as **true or false**, whereas agricultural systems often need **probabilities, confidence values, fuzzy values, and temporal information**.

A practical system could combine FOPC with:

- **Fuzzy logic** for gradual soil-moisture conditions.
- **Probabilistic reasoning** for uncertain sensor data.
- **Machine learning** for prediction.
- **Temporal reasoning** for changing weather and soil conditions.

Thus, predicate logic is useful as a rule-based foundation but should be combined with uncertainty-handling techniques.

3. Word Sense Disambiguation in a Travel Recommendation System

1. Determining the intended senses

Query	Intended Sense	Evidence
Amazon tour	Amazon rainforest	User viewed rainforest trekking packages
Safari booking	Wildlife tour	User selected wildlife resort packages
Java vacation	Java island/travel destination	User viewed hotels in Bali and Java
Cruise package	Ship journey	User viewed Mediterranean ship packages
Safari download for Windows	Web browser	User clicked a browser download page

The system should use both the query and subsequent user behaviour to determine the intended meaning.

2. Lexical and contextual cues

Amazon

- **Tour, trekking, rainforest, packages** → Amazon rainforest
- **Shopping, AWS, cloud, Prime** → Amazon company

Safari

- **Booking, wildlife, resort, animals** → wildlife safari
- **Download, Windows, browser, Apple** → Safari web browser

Java

- **Vacation, hotels, Bali, Indonesia, tourism** → Java island
- **Programming, code, JVM, developer** → Java programming language

Cruise

- **Mediterranean, ship, cabin, package, travel** → cruise journey
- **software, project, development** → software/project named Cruise

Thus, the meaning is determined by the **surrounding lexical context and user behaviour**, rather than the ambiguous word alone.

3. Why interpretation differs between users

The same word can have different meanings depending on:

1. **Query context**
2. **Previous searches**
3. **User interests**
4. **Geographical context**
5. **Click behaviour**
6. **Browsing history**
7. **Current task or session**

For example:

“Safari booking”

strongly suggests a wildlife trip.

But:

“Safari download for Windows”

strongly indicates the browser.

Therefore, WSD should be **context-sensitive and user-sensitive** rather than assigning one permanent meaning to a word.

4. Industrial-scale WSD strategy

A practical WSD architecture can be designed as:

Query → Context Extraction → Candidate Senses → Embedding Model → User History → Click Behaviour → Sense Ranking → Recommendation

Step 1: Query processing

Extract:

- Keywords
- Named entities
- Surrounding words
- Query intent
- Location information

Step 2: Generate candidate senses

For **Java**, generate:

- Java island
- Java programming language

Step 3: Context embeddings

Use Transformer-based embeddings to represent the query and compare it with representations of candidate senses.

For example:

“Java vacation hotels”

will have greater semantic similarity to **Java island** than Java programming.

Step 4: User-history features

Consider previous searches and viewed products. If a user frequently searches for hotels and flights, a travel-related interpretation should receive a higher score.

Step 5: Click behaviour

Clicks provide strong behavioural evidence.

For example:

“Safari download for Windows” → browser download clicked
should increase the probability of the **web browser** sense.

Step 6: Sense-ranking model

A ranking model can combine:

The sense with the highest score is selected.

Step 7: Continuous learning

The system can learn from:

- Click-through rates
- Booking conversions
- Search reformulations
- User feedback
- Abandoned recommendations

This makes the WSD system scalable and adaptive for large travel platforms.

4. Syntax-Driven Semantic Analysis in Legal Document Processing

1. Effect of syntactic structure on semantic roles

Syntactic structure provides important clues for assigning semantic roles, but **grammatical position and semantic role are not always identical**.

Active voice

“The supplier delivered the equipment to the buyer.”

- Subject = supplier
- Verb = delivered
- Object = equipment
- PP recipient = buyer

Semantic roles:

- Supplier = **Agent**
- Equipment = **Object/Theme**
- Buyer = **Recipient**

Another active sentence

“The buyer received the equipment from the supplier.”

Here:

- Buyer = grammatical subject
- Equipment = object
- Supplier = introduced through from

The verb **receive** changes the mapping of grammatical roles to semantic roles. Buyer is the **Recipient**, while supplier is the **Source/Agent**.

Passive voice

“The contract was terminated by the company.”

Although **contract** is the grammatical subject, it is not the semantic agent.

- Contract = **Patient/Theme**
- Company = **Agent**

The phrase “**by the company**” identifies the actual agent.

2. Incorrect semantic-role assignments

There are two important errors.

Error 1

“The buyer received the equipment from the supplier.”

The given interpretation says:

Buyer = Agent, Equipment = Object, Supplier = Recipient

This is incorrect.

A better interpretation is:

- **Buyer = Recipient**
- **Equipment = Theme/Object**
- **Supplier = Source/Agent**

The supplier provides the equipment, while the buyer receives it.

Error 2

“The contract was terminated by the company.”

The given interpretation says:

Contract = Agent, Company = Object

This is incorrect.

Correct interpretation:

- **Contract = Patient/Theme**
- **Company = Agent**

The contract undergoes the termination, while the company performs the action.

The interpretations for the first and third sentences are broadly correct.

3. Problems caused by confusing grammatical subject with semantic agent

A major problem in legal NLP is assuming:

Subject = Agent

This is not always true.

Consider:

“The contract was terminated by the company.”

The grammatical subject is **contract**, but the semantic agent is **company**.

If an automated legal system incorrectly labels the contract as the agent, it could extract the wrong information about **who performed the legal action**.

This can affect:

- Contract analysis
- Liability identification
- Responsibility extraction
- Legal event extraction
- Compliance analysis
- Case-law analysis
- Automated question answering

For example, a system might incorrectly conclude:

“The contract terminated the company.”

instead of:

“The company terminated the contract.”

Such an error could seriously distort the meaning of a legal document.

4. Recommended syntax-driven semantic architecture

A robust legal NLP system should use a multi-stage architecture:

Legal Text → Tokenization → Dependency/Constituency Parsing → Syntactic Analysis → Semantic Role Labeling → Coreference Resolution → Long-Distance Dependency Handling → Legal Event Extraction

Module 1: Syntactic parsing

Use dependency or constituency parsing to identify:

- Subject
- Verb
- Object
- Complements
- Prepositional phrases

Module 2: Voice detection

Determine whether the sentence is:

- Active
- Passive
- Causative
- Nominalized

For passive sentences, the system should specifically inspect **by-phrases** to identify potential agents.

Module 3: Semantic Role Labeling

Map syntactic constituents to semantic roles:

- Agent
- Patient/Theme
- Recipient
- Source
- Instrument
- Location
- Time

Module 4: Prepositional-phrase analysis

Different prepositions can indicate different roles.

For example:

“to the buyer” → Recipient

“from the supplier” → Source

“by the company” → Agent in passive construction

Module 5: Coreference resolution

Resolve references such as:

“The supplier delivered the equipment. **It** was inspected by the buyer.”

The system must determine what “**it**” refers to.

Module 6: Long-distance dependency handling

Legal sentences can contain deeply embedded clauses:

“The company that the supplier appointed delivered the equipment that the buyer requested.”

Dependency parsing and Transformer-based models can help connect related words across long distances.

Recommended architecture

A hybrid syntax-driven and neural semantic architecture would be most effective:

Dependency Parser + Semantic Role Labeler + Coreference Resolver + Transformer Model + Legal Ontology

This combines the structural interpretability of syntactic parsing with the contextual understanding of modern neural NLP models. It is particularly suitable for complex legal documents containing **active/passive constructions, prepositional phrases, nested clauses, and long-distance dependencies.**